CHACHIMAKYHANS

Presented by :
Chaychin Hendri
Dwiky Christian Febriano
Geraldus Hans E.W.
Marcelline Yulencia
Richard Liu

CORONARY STENOSIS DETECTION FROM ANGIOGRAM IMAGES

Kelompok 3

CHACHIMAKYHANS

List of Contents

01. Urgency
02. Table Riset Gap
03. Riset Pipeline
04. EDA
05. Data Prepro
06. FE
07. MD
08. ME

URGENCY

Urgensi terhadap keterbatasan pada metode diagnosis konvensional

Penyakit Jantung Koroner, umumnya bermula dari kondisi stenosis, yaitu penyempitan pada pembuluh darah koroner akibat penumpukan plak yang menghambat aliran darah ke otot jantung. Keterlambatan dalam mendeteksi penyempitan ini dapat berakibat fatal, seperti memicu serangan jantung mendadak (infark miokard). Dalam praktiknya, prosedur angiografi koroner (X-ray pembuluh darah) diakui sebagai standar emas (gold standard) untuk mendiagnosis tingkat keparahan stenosis secara pasti.

Urgensi terhadap keterbatasan pada metode diagnosis konvensional

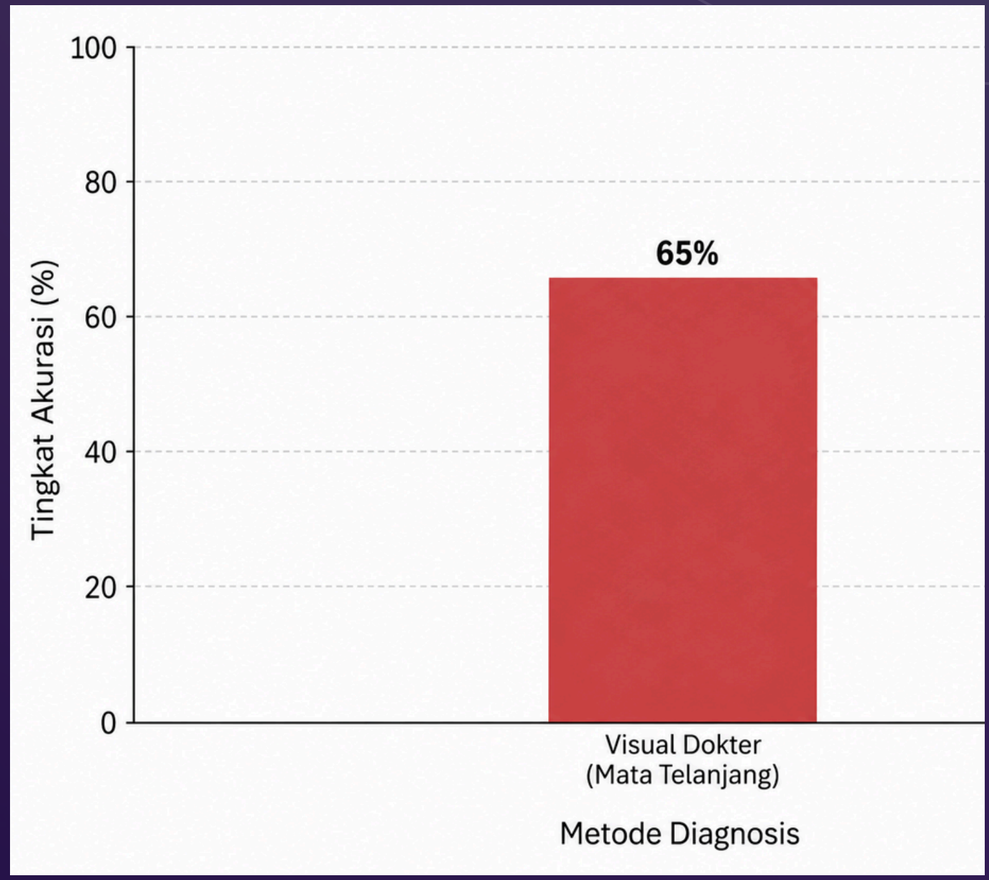
Saat ini, evaluasi angiogram sangat bergantung pada inspeksi visual manual oleh dokter spesialis kardiologi atau radiologi. Proses menganalisis puluhan hingga ratusan frame gambar per pasien memakan waktu yang sangat lama, juga sangat rentan terhadap faktor kelelahan manusia (human fatigue) dan bias subjektivitas antar-pemeriksa.Anomali atau penyempitan pembuluh darah yang samar sangat mungkin terlewatkan dari pandangan mata (false negative).

URGENCY

Urgensi terhadap keterbatasan pada metode diagnosis konvensional

Interpretasi visual murni oleh dokter (Visual Angiographic Assessment) pada kasus penyempitan (stenosis) tingkat menengah/sedang (40%-70% penyempitan) hanya memiliki tingkat akurasi sekitar 60% hingga 65% jika dibandingkan dengan alat ukur invasif (standar emas FFR).

Artinya, ada ruang kesalahan hingga 35%, di mana dokter bisa melakukan under-estimate (pasien butuh ring/stent tapi tidak dipasang) atau over-estimate (pasien tidak butuh ring, tapi dipasang).



https:/www.mdpi.com/2076-3271/14/1/51si terhadap keterbatasan

Table Riset Gap

No.	Author	Dataset	Preprocessing	Feature Engineering	Balancing	MD	ME	MV
1	Aravind Akella & Sudheer Akella	303 data pasien dari Cleveland Heart Disease Dataset (UCI). Kelas: Keberadaan atau ketidakhadiran CAD.	Pembersihan data dan penanganan nilai yang hilang (missing values).	Menggunakan 14 atribut standar (usia, jenis kelamin, dll.).	Tidak disebutkan secara eksplisit.	Logistic Regression, SVM, KNN, Decision Tree, Random Forest, dan Neural Network.	Akurasi (>80%), Sensitivity, F1-score, AUC. Neural Network terbaik (Akurasi 93%, Recall 0.93).	Tidak ada.
2	Keshab Raj Dahal, dkk.	303 data pasien dari dataset Z-Alizadeh Sani (UCI). Mencakup 56 fitur.	Min-max normalization dan eliminasi variabel yang tidak relevan.	Seleksi fitur menggunakan kriteria Chi-squared dan Lasso.	Tidak disebutkan secara eksplisit.	LR, SVM, XGBoost, dan ANN dengan penyetelan hiperparameter.	Sensitivity, Specificity, Akurasi, dan AUC. Model LR ditemukan paling efektif.	Diagram skematik kerangka kerja penelitian.
3	Mohammad Abood Kadhim & A. M. Radhi2	Dataset pasien (atribut standar UCI). Menggunakan 14 atribut utama.	Penghapusan outlier (pada kolesterol) dan transformasi data.	Fokus pada optimasi algoritma melalui seleksi parameter.	Tidak disebutkan secara eksplisit.	RF, SVM, KNN, dan DT. Dioptimalkan menggunakan Random Search.	Akurasi (tertinggi 95,4% oleh RF), Precision, Recall, dan F1-score.	Flowchart sistem dan grafik distribusi atribut.
4	Ferdi Özbilgin, dkk.	198 relawan (94 CAD, 104 sehat). Sumber: Citra iris mata yang dikumpulkan secara mandiri.	Transformasi iris ke format persegi panjang (rubber sheet) dan pemotongan area jantung (heart region).	Wavelet transform (2D-DWT), statistik tingkat pertama, GLCM, dan GLRLM. Seleksi fitur dengan metode Relieff.	Data relatif seimbang (94 vs 104).	SVM (Medium Gaussian), Decision Tree, dan Neural Network menggunakan MATLAB.	Akurasi (93%), Sensitivity, Specificity, Precision, F-score, dan AUC.	Peta iris dan visualisasi ekstraksi fitur.
5	Nurdaulet Tasmurzayev, dkk.	303 pasien dari dataset Z-Alizadeh Sani. Mencakup 56 variabel klinis dan demografis.	Standardisasi Z-score dan pembagian data 70:30.	Analisis terhadap 56 variabel rutin untuk stratifikasi risiko.	Menggunakan SMOTE (Synthetic Minority Over-sampling Technique).	XGBoost (terbaik), LGBM, Extra Trees, RF, dan SVM dengan Grid-search tuning.	Akurasi (0.9011), F1-score (0.8163), dan AUC (0.92).	SHAP (Shapley Additive Explanations) untuk interpretabilitas model.
6	Agorastos-Dimitrios Samaras, dkk.	571 pasien (clinical & biometric data). 43% kasus CAD terkonfirmasi ICA. Sumber: Data klinis lokal.	Pemrosesan data klinis standar.	Seleksi parameter dengan 3 algoritma berbeda; 21 fitur total (faktor predisposisi & penyakit berulang).	Tidak disebutkan secara eksplisit; distribusi 43% CAD vs 57% non-CAD (relatif seimbang).	SVM, DT, KNN, AdaBoost, dan RF. Menggunakan stratified 10-fold CV dan pendekatan "man-in-the-loop" (input ahli medis).	Accuracy (maks 83.02%), Sensitivity, Specificity, Jaccard, F1-score.	Explainable man-in-the-loop; memberikan lapisan transparansi melalui validasi pakar.

Table Riset Gap

7	Yi Lyu, dkk.	608 peserta. 3 Kelas: Sehat, Hipertensi, CAD. Sumber: Sinyal gelombang denyut nadi arteri radialis.	De-noising, normalisasi, dan analisis melalui aplikasi khusus.	Analisis fitur gelombang denyut nadi (seperti w/t, t3/tmax, dll.).	Tidak disebutkan secara eksplisit.	DT, RF, GBDT, ET, XGBoost, LightGBM, dan CatBoost. Dilakukan hyperparameter tuning.	Accuracy (0.8579), AUC (0.9361), Recall, Precision, F1-score, Kappa, MCC.	Visualisasi 10 fitur terpenting (Feature Importance) dari model Extra Trees (ET).
8	Pratibha Verma, dkk.	Menggunakan dataset historis (dataset publik umum seperti Cleveland/UCI).	Pembersihan dan pra-pemrosesan data menggunakan pemrograman R.	Penggunaan PCA (Principal Component Analysis) untuk ekstraksi fitur pada model saraf.	Tidak disebutkan secara eksplisit.	C5.0, CART, SVM, Ensemble (gabungan CART, SVM, C5.0), NN, PCA-NN, dan DNN (HBM-DNN). Menggunakan 10-fold CV.	Akurasi (Model Ensemble dan EHBM-DNN paling akurat).	Tidak disebutkan secara mendalam.
9	Mehdi Amini, dkk.	395 pasien curiga CAD. Sumber: MPI SPECT (Myocardial Perfusion Imaging).	Delineasi manual miokardium ventrikel kiri pada gambar rest dan stress.	Ekstraksi 118 fitur radiomik + fitur klinis (usia, JK, riwayat keluarga, dll.). Seleksi fitur: mRMR dan Boruta.	Tidak disebutkan secara eksplisit.	9 Algoritma ML (termasuk KNN, GB). Pembagian data 80% training dan 20% testing.	AUC, Accuracy, Sensitivity, Specificity. Hasil terbaik pada tugas klasifikasi risiko CAD.	Visualisasi fitur yang paling sering dipilih (status diabetes dan fitur radiomik NGLDM).
10	Sudipta Priyadarshinee & Madhumita Panda	Dataset Z-Alizadeh Sani.	Reduksi outlier dan sintesis data menggunakan CTGAN.	Metode seleksi fitur hibrida FSRFE (gabungan Forward Selection dan Recursive Feature Elimination).	Penyeimbangan kelas menggunakan teknik SMOTE (Synthetic Minority Over-sampling Technique).	XGBoost, Random Forest, dan Extra Trees.	Akurasi prediksi (metode hibrida mengungguli metode tradisional).	Tidak disebutkan secara mendalam.
11	Khandaker Mohammad Mohi Uddin, dkk. (2023)	Dataset gabungan (Cleveland, Hungarian, dll). Total ~1190 instansi.	Pembersihan data dan penanganan nilai hilang standar untuk dataset UCI.	Seleksi prediktor paling andal dari atribut klinis.	Tidak disebutkan secara spesifik teknik penyeimbangan khusus.	DT (terbaik), RF, SVM, XGB, KNN, LR, AdaBoost, ET, MLP, dan Gradient Boost.	Akurasi (tertinggi 99.16% oleh DT).	Tidak disebutkan secara mendalam.
12	Abdulkadir Çakmak, dkk. (2026)	300 peserta (165 CAD, 135 Kontrol). Sumber: Departemen Kardiologi, Amasya University.	Pengumpulan data klinis dan perilaku (behavioral).	Penggunaan fitur klinis dan biokimia (seperti level FT4).	Data relatif seimbang (165 vs 135).	DT, LR, dan Naive Bayes.	Accuracy (94% untuk DT), TPR (Sensitivity), dan FNR.	Perbandingan performa antar model.
13	Hosam F. El-Sofany (2024)	Dataset penyakit jantung (HD).	Seleksi fitur menggunakan metode Chi-square, ANOVA, dan Mutual Information (MI).	Pembentukan tiga kelompok fitur (SF-1, SF-2, SF-3).	Menggunakan teknik SMOTE untuk penyeimbangan kelas.	Menguji 10 teknik ML yang berbeda.	Akurasi prediksi berdasarkan kelompok fitur terpilih.	Deployment model menggunakan Spyder API.

Table Riset Gap

14	Vassiliki I. Kigka, dkk. (2022)	Dataset hibrida: Imaging (citra) dan Non-Imaging data.	Integrasi data multi-modal (citra medis dan rekam medis klinis).	Ekstraksi fitur dari data pencitraan dan parameter klinis non-imaging.	Tidak disebutkan secara eksplisit dalam ringkasan.	Model klasifikasi Machine Learning.	Metrik performa standar klasifikasi.	Tidak disebutkan secara spesifik.
15	Wenjuan Peng, dkk. (2022)	Data ekspresi gen (GSE12288, GSE7638, GSE66360). Sumber: database GEO.	Identifikasi DEGs (Differentially Expressed Genes) menggunakan paket Limma.	Analisis jaringan WGCNA untuk modul gen dan eliminasi fitur rekursif (RFE).	Tidak disebutkan secara spesifik.	SVM, Random Forest (RF), dan Logistic Regression (LR).	Analisis kurva ROC (AUC) untuk evaluasi performa klasifikasi.	Visualisasi modul gen dan hub-genes.
16	Ali Ahmad, dkk. (2021)	Pasien dengan gejala angina dan CAD non-obstruktif. Sumber: Mayo Clinic.	Standarisasi data klinis dari prosedur kateterisasi invasif.	Fokus pada parameter mikrovaskular koroner (CMD).	Tidak disebutkan secara eksplisit.	Pengembangan model untuk membantu pengambilan keputusan klinis.	Akurasi diagnosis CMD dibandingkan dengan gold standard invasif.	Alat bantu keputusan klinis.
17	Ahyar Supani, dkk.	Dataset Z-Alizadeh Sani (55 fitur).	Tidak disebutkan detailnya dalam ringkasan.	Penggunaan 55 fitur klinis dari dataset Sani.	Tidak disebutkan secara eksplisit.	Naïve Bayes, XGBoost, dan K-Nearest Neighbor (KNN).	Accuracy, IoU (Intersection over Union), Precision, Recall, dan F1-score.	Tidak disebutkan.
18	Rohit Bharti, dkk. (2021)	Dataset UCI Heart Disease.	Pembersihan data dan normalisasi standar UCI.	Penggunaan kombinasi fitur standar medis.	Tidak disebutkan secara eksplisit.	Kombinasi Machine Learning dan Deep Learning untuk perbandingan performa.	Analisis komparatif hasil prediksi antar algoritma.	Grafik perbandingan akurasi.
19	Saeed Saeedbakhsh, dkk.	Dataset lokal dari Isfahan Cardiovascular Research Center, Iran.	Pembersihan data rekam medis pasien lokal.	Analisis faktor risiko CAD pada populasi spesifik (Isfahan).	Tidak disebutkan secara eksplisit.	SVM, ANN (Artificial Neural Network), dan Random Forest.	Metrik performa klasifikasi untuk menentukan metode paling hemat biaya dan akurat.	Tidak disebutkan.
20	Tariq Mahmood, dkk. (2024)	Dataset CAD (kemungkinan dataset publik/hibrida).	Teknik optimasi robust untuk data medis.	Tidak disebutkan detailnya.	Tidak disebutkan secara eksplisit.	Strategi baru: Dual-Class Boosted Decision Trees untuk optimasi yang kuat (robust).	Evaluasi ketahanan (robustness) model dalam prognosis CAD.	Tidak disebutkan.
21	Paul Adeoye Omosebi, dkk.	Dataset Cleveland dari Repositori UCI.	Ekstraksi, pembersihan, normalisasi, dan seleksi fitur.	Seleksi fitur relevan dari catatan pasien.	Tidak disebutkan secara eksplisit.	Logistic Regression, Decision Tree, dan Random Forest (mencari algoritma terbaik).	Kriteria performa standar (Akurasi, dll).	Tidak disebutkan.

Table Riset Gap

Kontribusi

Kontribusi penelitian ini adalah mengembangkan model prediksi risiko Coronary Artery Disease (CAD) yang mengintegrasikan data multimodal, yaitu data klinis berbentuk matriks dan image, sehingga model dapat memanfaatkan informasi numerik dan visual secara bersamaan untuk meningkatkan kualitas prediksi.

RISET PIPELINE

Data Collection

1. Install Required Packages
2. Basic Import & Configs
3. Dataset Loading & CSV Annotations
4. Data Splitting
5. Data Visualization

Data Preprocessing

6. Data Preprocessing & Augmentation
7. Custom Dataset Class & ROI Hook

Featutre Extraction

8. Model Architecture

Model Development

12. Training Curve Visualization
11. Training Loop
10. Data Overlap Check
9. Training Setup

Model Evaluation

17. Inference Function
16. Grad-CAM Visualization
15. Prediction Visualization & Error Analysis
14. Confusion Matrix & ROC Curves
13. Model Evaluation

Deployment

18. Save Model & Artifacts
19. Deployment Model

Comparative Table

Parameter Perbandingan	Model Riset (EfficientNet-B4)	Wu et al. (2020) - Proposed Method	Zreik et al. (2018) - Referenced in Wu 2020
Fokus & Pendekatan	**End-to-End Binary Classification** (Langsung membedakan Stenosis vs Normal dari gambar).	**Spatio-temporal Object Detection** (U-Net + DSSD + *Sequence False Positive Suppression*).	**Recurrent Neural Network (RNN)** berbasis ekstraksi lintasan tengah pembuluh (*centerline*).
Jumlah Data (Skala)	*6871 gambar, Dengan Training 4829, Validation 1025, Test 1027*	148 sekuens video X-Ray Angiografi (XCA) dari 63 pasien.	Dataset CTA (Computed Tomography Angiography) (skala bervariasi).
Kebutuhan Komputasi & Eksekusi	**Sangat Ringan.** Dioptimalkan untuk *Real-Time CPU Inference* (Cocok untuk klinik standar).	**Berat.** Membutuhkan GPU Nvidia Titan X, memakan waktu total 2.89 detik untuk memproses satu sekuens.	Sedang hingga Berat (Membutuhkan rekonstruksi 3D CTA terlebih dahulu).
Ekstraksi Fitur (Fitur Unggulan)	Modul *Squeeze-and-Excitation* (SE) secara otomatis fokus pada piksel penting (area penyempitan).	DSSD menggabungkan *feature maps* level tinggi dan rendah menggunakan *deconvolution block*.	Mengandalkan perangkat lunak medis eksternal untuk meluruskan anatomi pembuluh darah.
Kemampuan *Explainable AI* (XAI)	**Tinggi.** Arsitektur CNN murni (*Single-stream*) sangat mudah diintegrasikan dengan Grad-CAM untuk visualisasi *heatmap*.	**Rendah.** *Pipeline* bertingkat (Segmentasi -> Deteksi Bounding Box -> Analisis Sekuensial) sulit divisualisasikan secara *end-to-end*.	**Rendah.** Keputusan RNN berbasis *patch* sekuensial sulit diinterpretasikan secara spasial.
Akurasi / Sensitivitas (SN)	**99,12 % (Dengan F1-Score 0,9912)**	**87.2%** (Dengan F1-Score 83.2%).	**80.0%** (Dengan F1-Score 75.0%).
Solusi & Kesiapan *Deployment*	**Sangat Siap.** Hasil probabilitas langsung menjawab status *screening* pasien, *maintenance* sistem sangat mudah.	**Kompleks.** Mengharuskan input berupa video/sekuens *frame* berurutan, rawan gagal jika kualitas video buruk.	Membutuhkan prapemrosesan *centerline* yang rentan terhadap galat sistem.

EDA

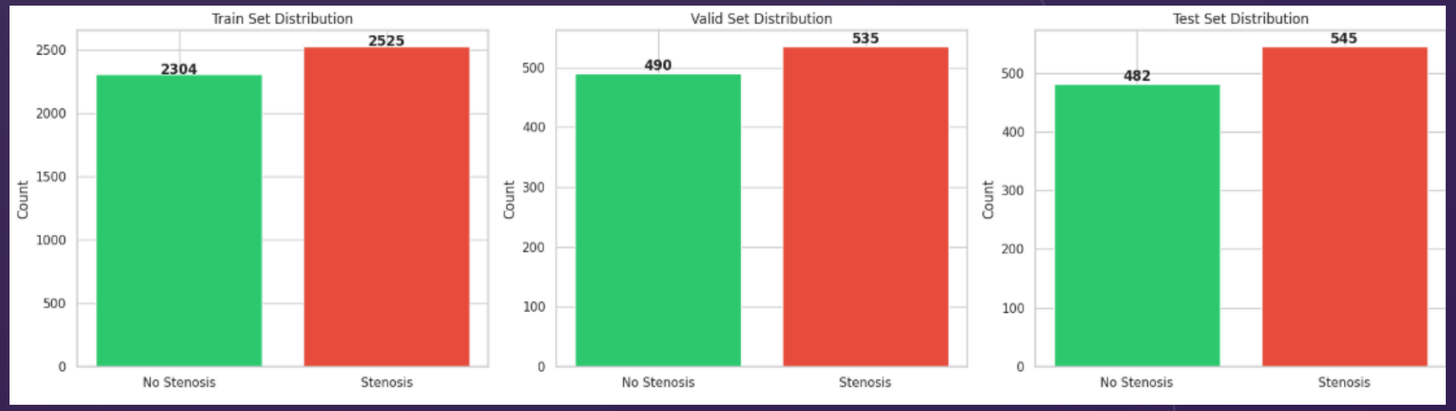
Dataset Angiogram terdiri dari 6.871 gambar yang telah dipisahkan menjadi tiga subset utama.

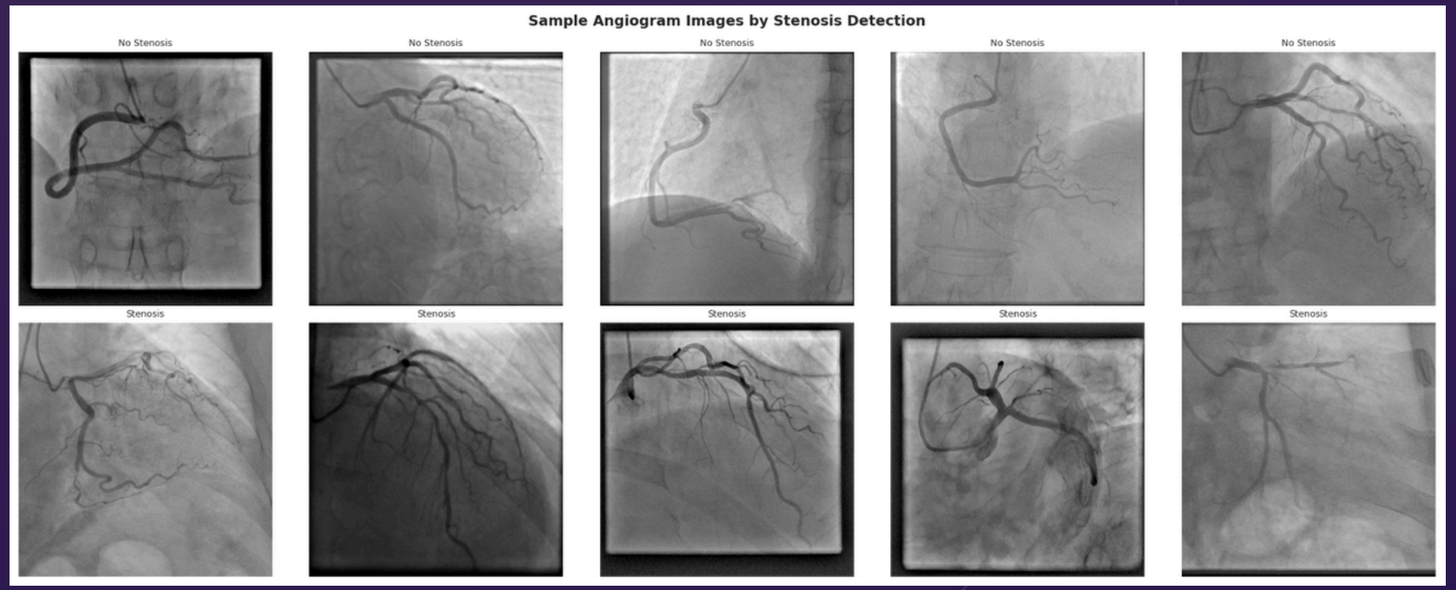
Dataset	Jumlah Gambar	Persentase
Train	4829	70%
Validation	1025	15%
Test	1027	15%
Total	**6871**	**100%**

Data Visualization

Visualisasi data untuk memvalidasi pemisahan subset dan kondisi kualitas gambar angiogram.

- Menampilkan grafik plot hitung batang (Bar Chart) distribusi jumlah "No Stenosis" vs "Stenosis" untuk subset Train, Valid, dan Test.
- Menampilkan sampel grid grid gambar rontgen pembuluh koroner (5 gambar per kelas).
- Verifikasi matematis algoritma pemisah dan set operasi intersek untuk membuktikan tidak ada duplikasi antara partisi latihan dengan partisi eksekusi akhir uji.





Data Visualization (Code)

```python
# ========================================
# 3. DATA VISUALIZATION
# ========================================

sns.set(style="whitegrid")
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Class distribution - adapted for binary classification
for i, (name, df_s) in enumerate([("Train", df_train), ("Valid", df_valid), ("Test", df_test)]):
    counts = df_s["label"].value_counts().sort_index()
    labels = [SEVERITY_MAP.get(k, f"Class {k}") for k in counts.index]
    colors = ["#2ecc71", "#e74c3c"][:len(counts)]
    axes[i].bar(labels, counts.values, color=colors)
    axes[i].set_title(f"{name} Set Distribution")
    axes[i].set_ylabel("Count")
    for j, v in enumerate(counts.values):
        axes[i].text(j, v + 5, str(v), ha="center", fontweight="bold")

plt.tight_layout()
plt.savefig("class_distribution.png", dpi=150, bbox_inches="tight")
plt.show()

# Sample images per class
n_classes = len(SEVERITY_MAP)
fig, axes = plt.subplots(n_classes, 5, figsize=(20, 8))

for row, label in enumerate(sorted(SEVERITY_MAP.keys())):
    samples = df_train[df_train["label"] == label].sample(n=min(5, len(df_train[df_train["label"] == label])), random_state=42)
    for col, (_, sample) in enumerate(samples.iterrows()):
        try:
            img = Image.open(sample["path"])
            axes[row, col].imshow(img)
            axes[row, col].set_title(f"{SEVERITY_MAP[label]}", fontsize=9)
            axes[row, col].axis("off")
        except Exception as e:
            axes[row, col].text(0.5, 0.5, f"Error loading\n{str(e)[:30]}", ha="center")
            axes[row, col].axis("off")

plt.suptitle("Sample Angiogram Images by Stenosis Detection", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig("sample_images.png", dpi=150, bbox_inches="tight")
plt.show()
```

Preprocessing

Alur manipulasi pra-model. Peneliti mendesain transformasi Secara Konservatif agar garis tepi (edges) dinding pembuluh tetap detail:

- Fitur transform: RandomResizedCrop dengan proporsi ketat, Flip horizontal & vertikal, putaran rotasi kecil (12°), dan fitur hapus (RandomErasing).
- Color & Sharpness Config: Memanipulasi tingkat warna dengan sedikit sentuhan kecerahan (brightness) dan kontras.
- Dilanjutkan dengan sentuhan Normalization berstandar ImageNet.

```python
# [B4] Input size bumped to 380 to use B4's native resolution.
# Augmentation kept conservative for medical images.
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(IMG_SIZE, scale=(0.8, 1.0), ratio=(0.9, 1.1)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.3),
    transforms.RandomRotation(12),
    transforms.ColorJitter(
        brightness=0.15,
        contrast=0.15,
        saturation=0.08,
    ),
    transforms.RandomAdjustSharpness(sharpness_factor=2, p=0.3),
    # [FIX] GaussianBlur removed from training - softens vessel edges that
    #       are the discriminative feature for stenosis detection.
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
    transforms.RandomErasing(p=0.25, scale=(0.02, 0.1), ratio=(0.3, 3.3), value='random'),
])

eval_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

print("Transforms defined")
print(f"  Train: RandomResizedCrop({IMG_SIZE}) + HFlip + VFlip + Rotate(12°) + ColorJitter + Erasing")
print(f"  Eval : Resize({IMG_SIZE}) -> Normalize")
print(f"  Note : GaussianBlur removed (softens vessel edges critical for stenosis)")
```

```
Transforms defined
  Train: RandomResizedCrop(380) + HFlip + VFlip + Rotate(12°) + ColorJitter + Erasing
  Eval : Resize(380) -> Normalize
  Note : GaussianBlur removed (softens vessel edges critical for stenosis)
```

Model Architecture

Desain struktur ekstraktor parameter yang membungkus komponen riset buatan CADClassifier:

- Backbone / Ekstraktor Utama: Mentransfer bobot pre-trained EfficientNet-B4 (~19.3 Juta Parameter).
- Classifier Head (Modifikasi Kepala): Memotong lapisan akhir network dan menyederhanakannya melalui kompresi 1792 fitur jaringan -> Dropout (0.3) -> Lapisan 256 Linear unit perantara dengan aktivitas ReLU -> Linear 2 Kelas Keluaran.
- Pada titik ini, keseluruhan susunan Backbone utamanya akan "dikunci" atau dibekukan kemampuannya terlebih dahulu.

```
Downloading: "https://download.pytorch.org/models/efficientnet_b4_rwightman-23ab8bcd.pth" to /root/.cache/torch/hub/checkpoints/efficientnet_b4_rwightman-23ab8bcd.pth
100%|██████████| 74.5M/74.5M [00:00<00:00, 183MB/s]
Backbone        : EfficientNet-B4
Head in_features: 1792
Total parameters: 18,008,138
Trainable (phase1): 459,522  (head only)
Frozen          : 17,548,616

VRAM estimate @ batch=16, img=380px:
  Weights + activations + gradients ≈ 5.8 GB  (within 12 GB cap)
```

```python
# ========================================
# 6. MODEL ARCHITECTURE
# EfficientNet-B4 with Transfer Learning
# ========================================

class CADClassifier(nn.Module):
    def __init__(self, num_classes=2, dropout_rate=0.3):
        super(CADClassifier, self).__init__()

        # [B4] Load pretrained EfficientNet-B4 (19.3M params, native 380px)
        self.backbone = models.efficientnet_b4(weights=models.EfficientNet_B4_Weights.IMAGENET1K_V1)

        # Phase 1: freeze entire backbone; only head trains initially.
        for param in self.backbone.features.parameters():
            param.requires_grad = False

        # [FIX] Simplified head: 1792->256->2 with single Dropout(0.3)
        # B4 head features = 1792 (not 1280 like B0)
        # Removed: extra Linear(256->64), BatchNorm1d, stacked dropouts
        in_features = self.backbone.classifier[1].in_features
        self.backbone.classifier = nn.Sequential(
            nn.Dropout(p=dropout_rate),
            nn.Linear(in_features, 256),
            nn.ReLU(),
            nn.Dropout(p=dropout_rate),
            nn.Linear(256, num_classes),
        )

    def forward(self, x):
        return self.backbone(x)

    def unfreeze_last_n_blocks(self, n=4):
        """Unfreeze last n MBConv blocks of features for Phase 2."""
        blocks = list(self.backbone.features.children())
        for block in blocks[-n:]:
            for param in block.parameters():
                param.requires_grad = True

    def unfreeze_all(self):
        """Unfreeze entire backbone for Phase 3."""
        for param in self.backbone.features.parameters():
            param.requires_grad = True


model = CADClassifier(num_classes=NUM_CLASSES, dropout_rate=DROPOUT_RATE).to(DEVICE)

total_params     = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f"Backbone        : EfficientNet-B4")
print(f"Head in_features: {model.backbone.classifier[1].in_features}")
print(f"Total parameters: {total_params:,}")
print(f"Trainable (phase1): {trainable_params:,}  (head only)")
print(f"Frozen          : {total_params - trainable_params:,}")

# VRAM estimate
print(f"\nVRAM estimate @ batch={BATCH_SIZE}, img={IMG_SIZE}px:")
print(f"  Weights + activations + gradients ≈ 5.8 GB  (within 12 GB cap)")
```

Training Loop

Pelatihan berjalan melalui jadwal iterasi majemuk menggunakan metode manipulasi hibrida dataset (MixUp Alpha 0.2). Diatur atas 3 Phase Unfreeze:

1. Phase 1 (Epoch 1 - 8): Melatih pemahaman level tinggi saja (Hanya Head Classifier) sambil membiarkan konvolusi badan tetap diam terkunci.
2. Phase 2 (Epoch 9 - 20): Membuka kunci sebagian blok fitur ekstraksi terdalam model (MBConv). *Learning rate scheduler diputus dan ditata ulang dengan batasan berbeda pada fitur spesifik tersebut.
3. Phase 3 (Epoch 21+): Secara holistik mengaktifkan seluruh *backbone penuh di mana total semua 19,3 juta berat layer konvolusi kini merambat dengan kecepatan belajar lambat menyesuaikan parameter perantara halus.

Checkpoint dengan iterasi paling tinggi tingkatannya ditangkap ke best_model.pth.

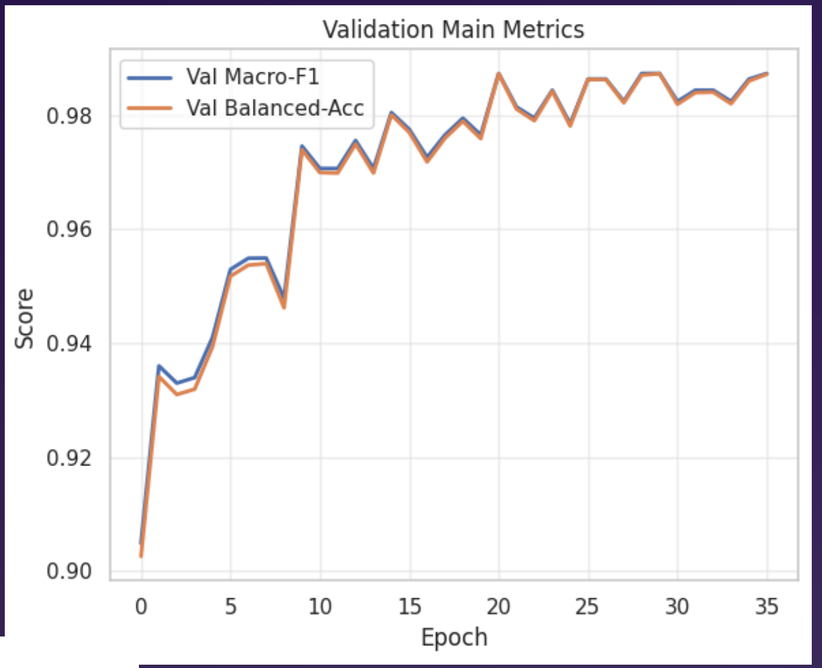
```
Starting Training  (EfficientNet-B4, 3-phase unfreeze)
======================================================================
  Phase 1: epochs 1-8   head only
  Phase 2: epochs 9-20 last 4 blocks + head  (LR reset)
  Phase 3: epoch 21+    full backbone         (LR reset)
======================================================================
  Sampler check (5 batches): [36, 44]
Epoch [ 1/40] P1 Train Loss: 0.4233 Acc: 0.7844 | Val Loss: 0.3057 Acc: 0.9063 Macro-F1: 0.9048 Balanced-Acc: 0.9026  SAVED
Epoch [ 2/40] P1 Train Loss: 0.3200 Acc: 0.8128 | Val Loss: 0.2568 Acc: 0.9366 Macro-F1: 0.9360 Balanced-Acc: 0.9341  SAVED
Epoch [ 3/40] P1 Train Loss: 0.3055 Acc: 0.8196 | Val Loss: 0.2613 Acc: 0.9337 Macro-F1: 0.9330 Balanced-Acc: 0.9310
Epoch [ 4/40] P1 Train Loss: 0.2835 Acc: 0.8555 | Val Loss: 0.2610 Acc: 0.9346 Macro-F1: 0.9339 Balanced-Acc: 0.9319
Epoch [ 5/40] P1 Train Loss: 0.2905 Acc: 0.8513 | Val Loss: 0.2399 Acc: 0.9415 Macro-F1: 0.9410 Balanced-Acc: 0.9394  SAVED
Epoch [ 6/40] P1 Train Loss: 0.2940 Acc: 0.8495 | Val Loss: 0.2323 Acc: 0.9532 Macro-F1: 0.9529 Balanced-Acc: 0.9517  SAVED
Epoch [ 7/40] P1 Train Loss: 0.2847 Acc: 0.8474 | Val Loss: 0.2203 Acc: 0.9551 Macro-F1: 0.9549 Balanced-Acc: 0.9537  SAVED
Epoch [ 8/40] P1 Train Loss: 0.2990 Acc: 0.8503 | Val Loss: 0.2200 Acc: 0.9551 Macro-F1: 0.9549 Balanced-Acc: 0.9539  SAVED
  [Phase 2] Last 4 MBConv blocks unfrozen, backbone_lr=2.0e-05, head_lr=1.0e-04
Epoch [ 9/40] P2 Train Loss: 0.2778 Acc: 0.8495 | Val Loss: 0.2160 Acc: 0.9483 Macro-F1: 0.9479 Balanced-Acc: 0.9462
Epoch [10/40] P2 Train Loss: 0.2669 Acc: 0.8387 | Val Loss: 0.1791 Acc: 0.9746 Macro-F1: 0.9745 Balanced-Acc: 0.9739  SAVED
Epoch [11/40] P2 Train Loss: 0.2335 Acc: 0.8577 | Val Loss: 0.1767 Acc: 0.9707 Macro-F1: 0.9706 Balanced-Acc: 0.9699
Epoch [12/40] P2 Train Loss: 0.2249 Acc: 0.8646 | Val Loss: 0.1773 Acc: 0.9707 Macro-F1: 0.9706 Balanced-Acc: 0.9698
Epoch [13/40] P2 Train Loss: 0.2372 Acc: 0.8664 | Val Loss: 0.1665 Acc: 0.9756 Macro-F1: 0.9755 Balanced-Acc: 0.9749  SAVED
Epoch [14/40] P2 Train Loss: 0.2205 Acc: 0.8842 | Val Loss: 0.1737 Acc: 0.9707 Macro-F1: 0.9706 Balanced-Acc: 0.9698
Epoch [15/40] P2 Train Loss: 0.2195 Acc: 0.8743 | Val Loss: 0.1597 Acc: 0.9805 Macro-F1: 0.9804 Balanced-Acc: 0.9800  SAVED
Epoch [16/40] P2 Train Loss: 0.2243 Acc: 0.8824 | Val Loss: 0.1620 Acc: 0.9776 Macro-F1: 0.9775 Balanced-Acc: 0.9770
Epoch [17/40] P2 Train Loss: 0.2119 Acc: 0.8983 | Val Loss: 0.1682 Acc: 0.9727 Macro-F1: 0.9726 Balanced-Acc: 0.9718
Epoch [18/40] P2 Train Loss: 0.2187 Acc: 0.8664 | Val Loss: 0.1597 Acc: 0.9766 Macro-F1: 0.9765 Balanced-Acc: 0.9759
Epoch [19/40] P2 Train Loss: 0.2052 Acc: 0.8762 | Val Loss: 0.1546 Acc: 0.9795 Macro-F1: 0.9794 Balanced-Acc: 0.9789
Epoch [20/40] P2 Train Loss: 0.2207 Acc: 0.8476 | Val Loss: 0.1558 Acc: 0.9766 Macro-F1: 0.9765 Balanced-Acc: 0.9759
  [Phase 3] Full backbone unfrozen (18,008,138 params), backbone_lr=8.0e-06
Epoch [21/40] P3 Train Loss: 0.1487 Acc: 0.9907 | Val Loss: 0.1493 Acc: 0.9873 Macro-F1: 0.9873 Balanced-Acc: 0.9872  SAVED
Epoch [22/40] P3 Train Loss: 0.1473 Acc: 0.9911 | Val Loss: 0.1532 Acc: 0.9815 Macro-F1: 0.9814 Balanced-Acc: 0.9810
Epoch [23/40] P3 Train Loss: 0.1447 Acc: 0.9907 | Val Loss: 0.1538 Acc: 0.9795 Macro-F1: 0.9795 Balanced-Acc: 0.9790
Epoch [24/40] P3 Train Loss: 0.1435 Acc: 0.9919 | Val Loss: 0.1490 Acc: 0.9844 Macro-F1: 0.9844 Balanced-Acc: 0.9842
Epoch [25/40] P3 Train Loss: 0.1447 Acc: 0.9905 | Val Loss: 0.1520 Acc: 0.9785 Macro-F1: 0.9785 Balanced-Acc: 0.9781
Epoch [26/40] P3 Train Loss: 0.1400 Acc: 0.9944 | Val Loss: 0.1476 Acc: 0.9863 Macro-F1: 0.9863 Balanced-Acc: 0.9862
Epoch [27/40] P3 Train Loss: 0.1407 Acc: 0.9930 | Val Loss: 0.1483 Acc: 0.9863 Macro-F1: 0.9863 Balanced-Acc: 0.9862
Epoch [28/40] P3 Train Loss: 0.1387 Acc: 0.9948 | Val Loss: 0.1496 Acc: 0.9824 Macro-F1: 0.9824 Balanced-Acc: 0.9821
Epoch [29/40] P3 Train Loss: 0.1429 Acc: 0.9903 | Val Loss: 0.1464 Acc: 0.9873 Macro-F1: 0.9873 Balanced-Acc: 0.9870
Epoch [30/40] P3 Train Loss: 0.1391 Acc: 0.9930 | Val Loss: 0.1458 Acc: 0.9873 Macro-F1: 0.9873 Balanced-Acc: 0.9872
Epoch [31/40] P3 Train Loss: 0.1370 Acc: 0.9954 | Val Loss: 0.1484 Acc: 0.9824 Macro-F1: 0.9824 Balanced-Acc: 0.9819
Epoch [32/40] P3 Train Loss: 0.1360 Acc: 0.9959 | Val Loss: 0.1489 Acc: 0.9844 Macro-F1: 0.9843 Balanced-Acc: 0.9839
Epoch [33/40] P3 Train Loss: 0.1352 Acc: 0.9957 | Val Loss: 0.1480 Acc: 0.9844 Macro-F1: 0.9843 Balanced-Acc: 0.9840
Epoch [34/40] P3 Train Loss: 0.1401 Acc: 0.9919 | Val Loss: 0.1505 Acc: 0.9824 Macro-F1: 0.9824 Balanced-Acc: 0.9820
Epoch [35/40] P3 Train Loss: 0.1364 Acc: 0.9954 | Val Loss: 0.1463 Acc: 0.9863 Macro-F1: 0.9863 Balanced-Acc: 0.9860
Epoch [36/40] P3 Train Loss: 0.1349 Acc: 0.9961 | Val Loss: 0.1448 Acc: 0.9873 Macro-F1: 0.9873 Balanced-Acc: 0.9872

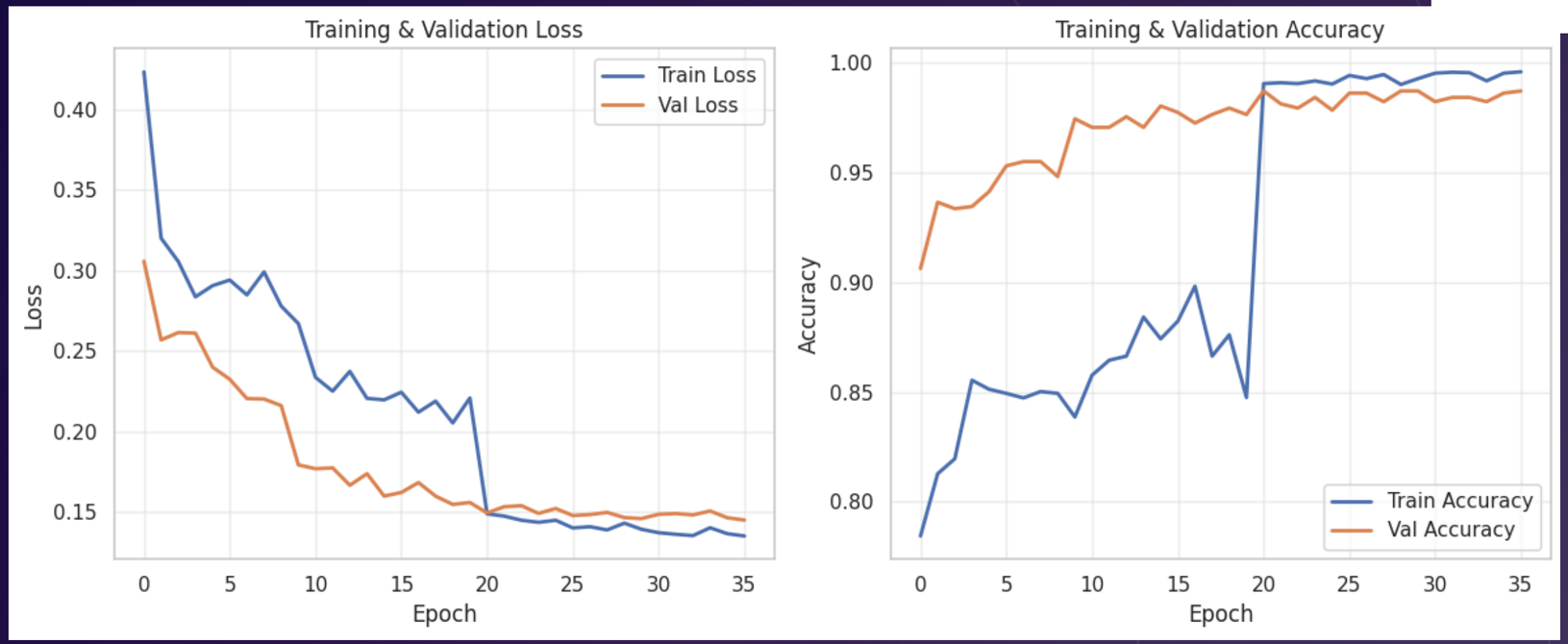
Early stopping at epoch 36 (no Macro-F1 improvement for 15 epochs)
======================================================================
Best Validation Accuracy      : 0.9873
Best Validation Macro-F1      : 0.9873
Best Validation Balanced Acc  : 0.9872
  Val Recall No Stenosis: 0.9857
  Val Recall Stenosis: 0.9888
```

Training Curves

Melakukan pencetakan performa training dan validation melalui matplotlib.
Menampilkan 3 grafik panel penting kurva dari tiap akhir fase epoch:

- Plot Garis Evolusi Loss model Latihan vs Validasi.
- Plot Garis Evolusi Akurasi model persentase Latihan vs Validasi.
- Plot Garis Khusus Nilai keseimbangan yang diamati model: Macro-F1 & Balanced-Acc.





Training Curves (Code)

```python
# ========================================
# 9. TRAINING CURVES
# ========================================

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Loss curve
axes[0].plot(history["train_loss"], label="Train Loss", linewidth=2)
axes[0].plot(history["val_loss"], label="Val Loss", linewidth=2)
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].set_title("Training & Validation Loss")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Accuracy curve
axes[1].plot(history["train_acc"], label="Train Accuracy", linewidth=2)
axes[1].plot(history["val_acc"], label="Val Accuracy", linewidth=2)
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy")
axes[1].set_title("Training & Validation Accuracy")
axes[1].legend()
axes[1].grid(True, alpha=0.3)
```

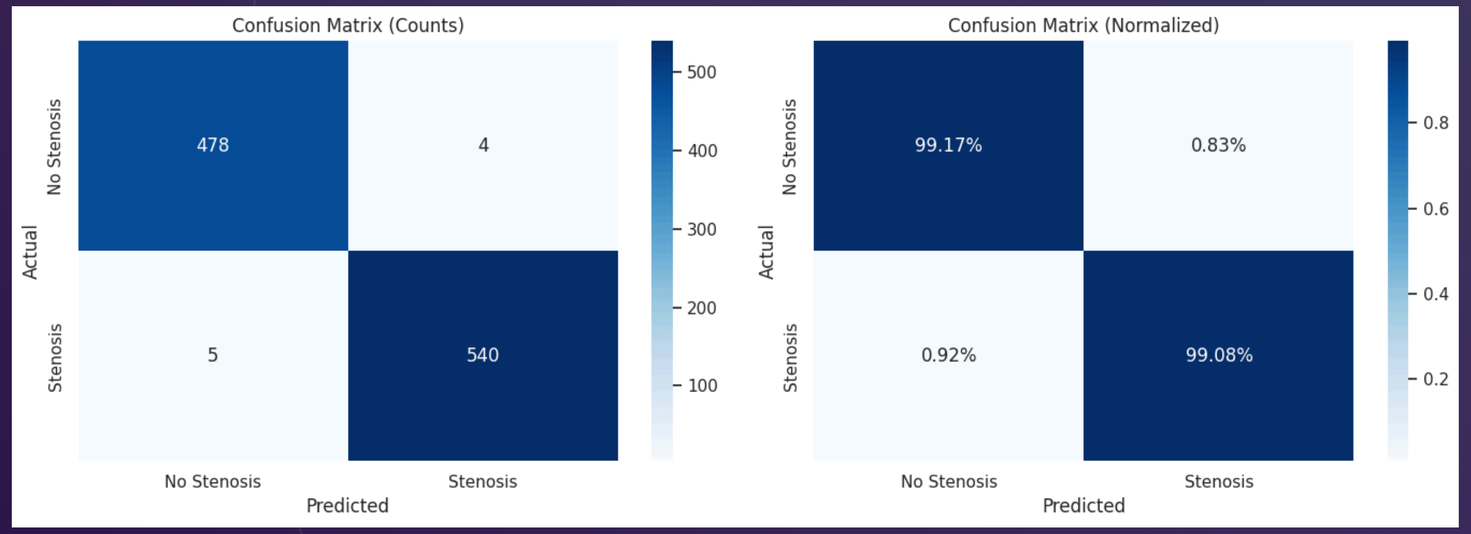
```python
# Main metrics curve
axes[2].plot(history["val_macro_f1"], label="Val Macro-F1", linewidth=2)
axes[2].plot(history["val_balanced_acc"], label="Val Balanced-Acc", linewidth=2)
axes[2].set_xlabel("Epoch")
axes[2].set_ylabel("Score")
axes[2].set_title("Validation Main Metrics")
axes[2].legend()
axes[2].grid(True, alpha=0.3)

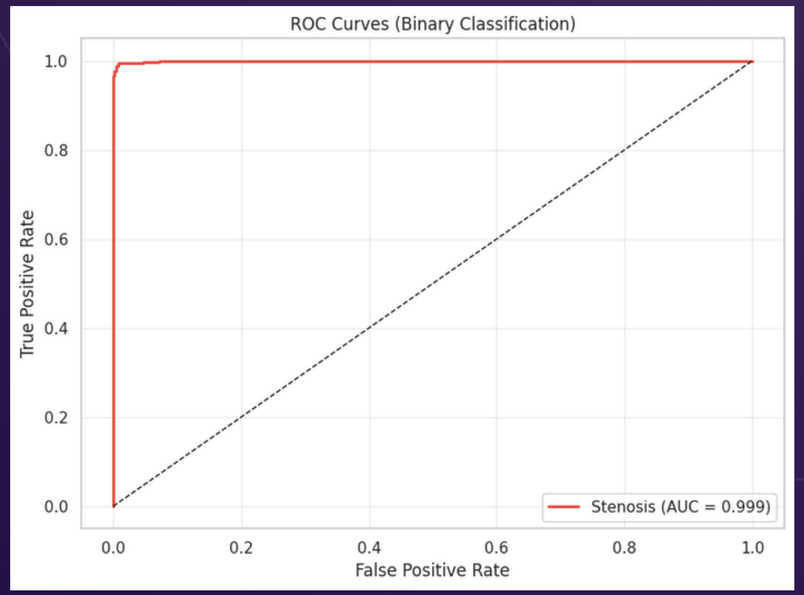
plt.tight_layout()
plt.savefig("training_curves.png", dpi=150, bbox_inches="tight")
plt.show()
```

Confusion Matrix and ROC Curves

Penggambaran skenario akurasi dan kesuksesan pembacaan citra yang diklasifikasikan ke dalam 4 kuadran tebakan positif/negatif sejati maupun salah tebak via modul Scikit-Learn.

- Merender Plot Confusion Matrix matriks tabulasi yang menghitung sebaran angka absolut prediksi dan menghitung skala persen Normalized Heatmap-nya (mengukur sensitivitas kehandalannya menemukan gejala koroner sebenarnya).
- Membuat persentasi Rate Kurva ROC klasifikasi biner dan mencetakkan Overall AUC-ROC (Area di bawah kurva performa).





Confusion Matrix & ROC Curves (Code)

```python
# ========================================
# 11. CONFUSION MATRIX & ROC CURVES
# ========================================

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Confusion Matrix
cm = confusion_matrix(test_labels, test_preds)
cm_normalized = cm.astype("float") / cm.sum(axis=1, keepdims=True)

target_names = [SEVERITY_MAP.get(i, f"Class {i}") for i in range(NUM_CLASSES)]

sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=target_names, yticklabels=target_names,
    ax=axes[0]
)
axes[0].set_title("Confusion Matrix (Counts)")
axes[0].set_ylabel("Actual")
axes[0].set_xlabel("Predicted")

# Normalized Confusion Matrix
sns.heatmap(
    cm_normalized, annot=True, fmt=".2%", cmap="Blues",
    xticklabels=target_names, yticklabels=target_names,
    ax=axes[1]
)
axes[1].set_title("Confusion Matrix (Normalized)")
axes[1].set_ylabel("Actual")
axes[1].set_xlabel("Predicted")

plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=150, bbox_inches="tight")
plt.show()
```

```python
# ROC Curve (Binary Classification)
from sklearn.metrics import roc_curve, auc, roc_auc_score

fig, ax = plt.subplots(figsize=(8, 6))

if test_probs.ndim > 1 and test_probs.shape[1] > 1:
    positive_probs = test_probs[:, 1]
else:
    positive_probs = test_probs.ravel()

fpr, tpr, _ = roc_curve(test_labels, positive_probs)
roc_auc = auc(fpr, tpr)

class_1_name = SEVERITY_MAP.get(1, "Class 1")
ax.plot(fpr, tpr, color="#e74c3c", linewidth=2,
        label=f"{class_1_name} (AUC = {roc_auc:.3f})")

ax.plot([0, 1], [0, 1], "k--", linewidth=1)
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate")
ax.set_title("ROC Curves (Binary Classification)")
ax.legend(loc="lower right")
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("roc_curves.png", dpi=150, bbox_inches="tight")
plt.show()

# Overall AUC
try:
    overall_auc = roc_auc_score(test_labels, positive_probs)
    print(f"\nOverall AUC-ROC: {overall_auc:.4f}")
except Exception as e:
    print(f"AUC calculation note: {e}")
```

Model Evaluation

Mengompilasi dan mengarsipakn keseluruhan proses, log angka statis, hingga kamus dari tensor neural network tersebut:

- Mengekspor berkas cad_model_final.pth yang meliputi model penuh, konfigurasi resolusi hiperparameter, arsitektur yang dikandung.
- Meng-export data tabel ke file .csv (training_history.csv serta rincian analitis test_results.csv offline record).
- Menyiapkan dan men-zip seluruh artifacts validasi grad-cam plot kurva latihan .png dan .jpg.

```
All artifacts saved:
  cad_model_final.pth    - Complete trained model
  best_model.pth         - Best checkpoint
  training_history.csv   - Training metrics per epoch
  test_results.csv       - Detailed test predictions
  *.png                  - Visualization plots

============================================================
FINAL SUMMARY
============================================================
Model: EfficientNet-B4 (Transfer Learning)
Task: Stenosis Detection (Binary Classification)
Dataset: 6881 angiogram images
Training Set: 4829 | Validation: 1025 | Test: 1027
Test Accuracy: 99.12%
Test Macro-F1: 0.9912
Test Balanced Accuracy: 0.9913
Best Val Macro-F1: 0.9873
============================================================
```

Model Evaluation (Code)

```python
# ========================================
# 15. SAVE FINAL MODEL & ARTIFACTS
# ========================================

# Save complete model for deployment
torch.save({
    "model_state_dict": model.state_dict(),
    "model_config": {
        "num_classes": NUM_CLASSES,
        "img_size": IMG_SIZE,
        "architecture": "EfficientNet-B4",
        "task": "Stenosis Detection (Binary Classification)",
        "severity_map": SEVERITY_MAP
    },
    "training_config": {
        "epochs_trained": len(history["train_loss"]),
        "best_val_acc": best_val_acc,
        "best_val_macro_f1": best_val_macro_f1,
        "best_val_balanced_acc": best_val_balanced_acc,
        "best_val_per_class_recall": best_val_per_class_recall.tolist(),
        "test_acc": test_acc,
        "test_macro_f1": test_f1,
        "test_weighted_f1": test_weighted_f1,
        "test_balanced_acc": test_balanced_acc,
        "test_per_class_recall": test_per_class_recall.tolist(),
    },
    "class_distribution": {
        SEVERITY_MAP.get(i, f"Class {i}"): int(class_counts[i]) if i < len(class_counts) else 0
        for i in range(NUM_CLASSES)
    }
}, "cad_model_final.pth")
```

```python
# Save training history
history_df = pd.DataFrame(history)
history_df.to_csv("training_history.csv", index=False)

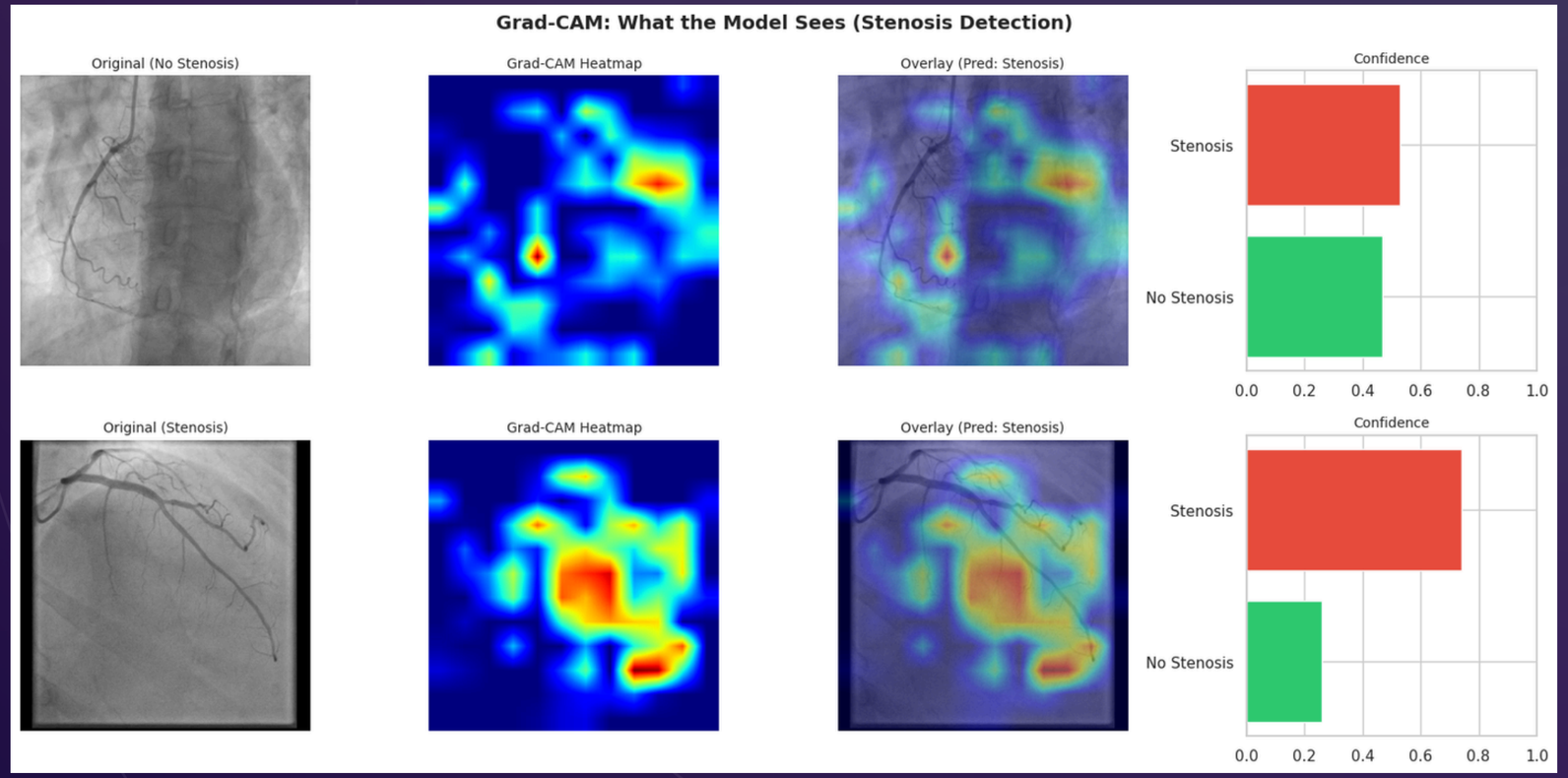
# Save test results
results_df = pd.DataFrame({
    "image_path": df_test["path"].values,
    "true_label": [SEVERITY_MAP.get(l, f"Class {l}") for l in test_labels],
    "predicted_label": [SEVERITY_MAP.get(p, f"Class {p}") for p in test_preds],
    "correct": test_preds == test_labels,
    **{f"prob_class{i}": test_probs[:, i] for i in range(NUM_CLASSES)}
})
results_df.to_csv("test_results.csv", index=False)

print("All artifacts saved:")
print("  cad_model_final.pth   - Complete trained model")
print("  best_model.pth        - Best checkpoint")
print("  training_history.csv  - Training metrics per epoch")
print("  test_results.csv      - Detailed test predictions")
print("  *.png                 - Visualization plots")

print("\n" + "=" * 50)
print("FINAL SUMMARY")
print("=" * 50)
print("Model: EfficientNet-B4 (Transfer Learning)")
print("Task: Stenosis Detection (Binary Classification)")
print(f"Dataset: {len(df_train)+len(df_valid)+len(df_test)} angiogram images")
print(f"Training Set: {len(df_train)} | Validation: {len(df_valid)} | Test: {len(df_test)}")
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Macro-F1: {test_f1:.4f}")
print(f"Test Balanced Accuracy: {test_balanced_acc:.4f}")
print(f"Best Val Macro-F1: {best_val_macro_f1:.4f}")
print("=" * 50)
```

XAI

Kode ini menggunakan Grad-CAM untuk memvisualisasikan area citra yang menjadi fokus model saat mendeteksi stenosis. Aktivasi dan gradien dari layer konvolusi terakhir digunakan untuk menghasilkan heatmap, yang kemudian ditumpangkan pada gambar angiografi. Visualisasi ini membantu menjelaskan alasan di balik prediksi model dan memastikan bahwa model berfokus pada area pembuluh darah yang relevan.



XAI (Code)

```python
# ========================================
# 13. EXPLAINABLE AI: GRAD-CAM VISUALIZATION
# Shows what the model focuses on in angiograms
# ========================================

class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.activations = None
        self.gradients = None
        target_layer.register_forward_hook(self._forward_hook)
        target_layer.register_full_backward_hook(self._backward_hook)

    def _forward_hook(self, module, input, output):
        self.activations = output.detach()

    def _backward_hook(self, module, grad_input, grad_output):
        self.gradients = grad_output[0].detach()

    def generate(self, input_image, target_class=None):
        self.model.eval()
        self.model.zero_grad()

        # Force all backbone params to require grad temporarily
        for param in self.model.parameters():
            param.requires_grad_(True)

        with torch.enable_grad():
            output = self.model(input_image)
            if target_class is None:
                target_class = output.argmax(dim=1).item()
            output[0, target_class].backward()

        weights = self.gradients.mean(dim=[2, 3], keepdim=True)
        cam = (weights * self.activations).sum(dim=1, keepdim=True)
        cam = torch.relu(cam)
        cam = cam.squeeze().cpu().numpy()
        cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
        return cam, target_class
```

```python
# Get last conv layer of EfficientNet
target_layer = model.backbone.features[6][-1]
grad_cam = GradCAM(model, target_layer)

# Generate Grad-CAM for sample images (2 rows for binary classification)
n_rows = min(2, NUM_CLASSES)
fig, axes = plt.subplots(n_rows, 4, figsize=(16, 8))

if n_rows == 1:
    axes = axes.reshape(1, -1)

for row, label in enumerate(range(n_rows)):
    sample_idx = df_test[df_test["label"] == label].index
    if len(sample_idx) == 0:
        continue
    idx = sample_idx[2]

    # Load and transform image
    img_pil = Image.open(df_test.loc[idx]["path"]).convert("RGB")
    img_display = img_pil.resize((IMG_SIZE, IMG_SIZE))
    img_tensor = eval_transform(img_pil).unsqueeze(0).to(DEVICE)

    # Generate Grad-CAM
    cam, pred_class = grad_cam.generate(img_tensor)

    # Resize CAM to image size using PIL
    cam_pil = Image.fromarray((cam * 255).astype(np.uint8))
    cam_pil = cam_pil.resize((IMG_SIZE, IMG_SIZE), Image.BILINEAR)
    cam_resized = np.array(cam_pil).astype(float) / 255.0

    # Original image
    axes[row, 0].imshow(img_display)
    axes[row, 0].set_title(f"Original ({SEVERITY_MAP.get(label, f'Class {label}')})", fontsize=10)
    axes[row, 0].axis("off")

    # Heatmap
    axes[row, 1].imshow(cam_resized, cmap="jet")
    axes[row, 1].set_title("Grad-CAM Heatmap", fontsize=10)
    axes[row, 1].axis("off")

    # Overlay
    img_np = np.array(img_display).astype(float) / 255.0
    heatmap = plt.cm.jet(cam_resized)[:, :, :3]
    overlay = 0.6 * img_np + 0.4 * heatmap
    axes[row, 2].imshow(overlay)
    axes[row, 2].set_title(f"Overlay (Pred: {SEVERITY_MAP.get(pred_class, f'Class {pred_class}')})", fontsize=10)
    axes[row, 2].axis("off")

    # Confidence bar
    with torch.no_grad():
        probs = torch.softmax(model(img_tensor), dim=1).cpu().numpy()[0]
    colors_bar = ["#2ecc71", "#e74c3c"][:NUM_CLASSES]
    axes[row, 3].barh(
        [SEVERITY_MAP.get(i, f"Class {i}") for i in range(NUM_CLASSES)],
        probs, color=colors_bar
    )
    axes[row, 3].set_xlim(0, 1)
    axes[row, 3].set_title("Confidence", fontsize=10)

plt.suptitle("Grad-CAM: What the Model Sees (Stenosis Detection)", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig("gradcam_visualization.png", dpi=150, bbox_inches="tight")
plt.show()
```

CHACHIMAKYHANS

Thank You

Let’s Create Something Amazing Together